Smart Health AI: Best Implementation Document

1. Problem Understanding

Primary Health Centres (PHCs) and Community Health Centres (CHCs) often track medicines, beds, doctor attendance, test availability, and patient load manually. This creates delays in identifying stock-outs, overcrowding, absent doctors, and under-resourced facilities. District administrators do not get real-time visibility, so intervention usually happens after service quality has already dropped.

The best solution is not a full hospital ERP. For a hackathon or prototype, the strongest implementation is an AI-powered district health operations platform that monitors PHCs/CHCs, predicts risk, recommends action, and explains decisions clearly.

Core value proposition:

> *"Observe live health-center status, predict shortages and surges, recommend redistribution, and help district officials act before services fail.*

2. Recommended Product Positioning

Brand the solution as a District Health Digital Twin or AI Health Command Center.

This framing is stronger than "dashboard" because it shows four clear capabilities:

1. Observe: Real-time monitoring of PHCs/CHCs.
2. Predict: AI forecasts medicine stock-outs, patient surges, and resource gaps.
3. Recommend: The system suggests transfers of medicines, doctors, beds, and patient referrals.
4. Act: District admins receive prioritized alerts and intervention plans.

3. User Roles

Health Center Staff

Staff should have a simple worker portal for daily updates:

- Update medicine stock and expiry dates.
- Update occupied beds.
- Mark doctor attendance.
- Update test availability.
- Enter daily patient count.
- Use voice input for faster data entry.

District Administrator

Admins should get an operations dashboard:

- View all PHCs/CHCs across the district.

- See critical alerts.
- Compare centers by health score.
- View map-based risk status.
- Review AI recommendations.
- Generate daily, weekly, and monthly reports.

AI Assistant

The AI assistant should:

- Explain which centers need attention.
- Summarize risk reasons.
- Recommend redistribution.
- Generate multilingual reports.
- Answer admin questions using structured database context.

4. Best Implementation Strategy

Use a hybrid system:

- Deterministic backend logic for critical decisions like thresholds, health scores, and basic alerts.
- Gemini for natural-language explanations, report generation, demand reasoning, and admin chatbot responses.
- Lightweight forecasting logic using historical data, with Gemini used to explain or refine the forecast.

This is better than making every feature depend on an LLM because the system remains predictable, cheap, faster to demo, and easier to debug.

5. Scope

Must-Have MVP

- Authentication with two roles: health worker and district admin.
- Worker portal for medicines, beds, doctors, tests, and patients.
- Admin dashboard with summary cards, charts, alerts, and map.
- Threshold-based alert engine.
- Health score for each PHC/CHC.
- Medicine stock-out prediction.
- Redistribution recommendations.
- AI assistant for district-level queries.
- Seeded demo dataset.

High-Impact Additions

- AI command center feed.
- Multilingual report generation in English, Hindi, and Gujarati.

- Voice input for health workers.
- PDF export for daily and weekly reports.
- Live demo flow where changing stock instantly triggers alerts, score changes, and map status updates.

Avoid in Version 1

- Full hospital ERP workflows.
- Real biometric attendance hardware.
- Paid SMS/WhatsApp APIs.
- Complex LSTM or deep learning models.
- Overly large patient-level medical records.
- Google Maps paid integration.

6. Recommended Tech Stack

```
Layer | Recommended Technology | Reason
Frontend | Next.js 15 + TypeScript + Tailwind CSS | Fast development, modern UI, easy deployment
Backend | FastAPI | Clean APIs, Python AI ecosystem
Database | Supabase PostgreSQL | Free tier, relational data, auth support
Authentication | Supabase Auth | Simple role-based login
AI | Google Gemini Flash / Flash Lite | Free-tier friendly, good for reasoning and summaries
Charts | Recharts | Simple dashboard visualizations
Maps | Leaflet + OpenStreetMap | Free and reliable
Reports | HTML-to-PDF or Python PDF library | Good enough for hackathon reporting
Notifications | In-app alerts | Avoids paid SMS dependency
Deployment | Vercel + Render or Cloud Run + Supabase | Free-tier friendly deployment
```

7. System Architecture

```
  Health Worker / District Admin
            |
            v
  Next.js Frontend
            |
            v
  FastAPI Backend
            |
            v
  Supabase PostgreSQL
            |
            v
  AI and Analytics Layer
            |
            +-- Alert Engine
            +-- Health Score Engine
            +-- Forecasting Service
            +-- Redistribution Optimizer
            +-- Gemini Assistant
            +-- Report Generator
```

Recommended backend responsibility:

- Validate updates from health workers.
- Store all operational data.
- Calculate alerts and health scores.
- Generate recommendations.
- Provide structured context to Gemini.
- Store AI outputs so the frontend can render them quickly.

Recommended frontend responsibility:

- Role-based interfaces.
- Worker data entry screens.
- Admin command center.
- Charts, maps, alert feed, and reports.
- Voice input capture using the browser Web Speech API.

8. Database Design

phcs

Stores each health center.

```
Field | Type | Notes
id | uuid | Primary key
name | text | PHC/CHC name
type | text | PHC or CHC
district | text | District name
address | text | Optional
latitude | numeric | Map marker
longitude | numeric | Map marker
created_at | timestamp | Default now
```

users_profile

Stores app-specific role data linked to Supabase Auth.

```
Field | Type | Notes
id | uuid | Supabase auth user id
full_name | text | User name
role | text | admin or worker
phc_id | uuid | Nullable for district admin
```

medicines

Stores stock data by PHC.

```
Field | Type | Notes
id | uuid | Primary key
phc_id | uuid | Foreign key
medicine_name | text | Example: Paracetamol
stock | integer | Current quantity
threshold | integer | Minimum safe stock
```

```
expiry_date | date | Optional
updated_at | timestamp | Last update
```

stock_movements

Tracks stock usage and transfers. This is important for forecasting.

```
Field | Type | Notes
id | uuid | Primary key
phc_id | uuid | Foreign key
medicine_name | text | Medicine
quantity_change | integer | Negative for usage, positive for restock
reason | text | usage, restock, transfer_in, transfer_out
created_at | timestamp | Event time
```

beds

```
Field | Type | Notes
id | uuid | Primary key
phc_id | uuid | Foreign key
total | integer | Total beds
occupied | integer | Occupied beds
updated_at | timestamp | Last update
```

doctors

```
Field | Type | Notes
id | uuid | Primary key
phc_id | uuid | Foreign key
doctor_name | text | Doctor name
specialization | text | Optional
active | boolean | Current posting status
```

doctor_attendance

```
Field | Type | Notes
id | uuid | Primary key
doctor_id | uuid | Foreign key
phc_id | uuid | Foreign key
date | date | Attendance date
status | text | present, absent, leave
time_in | time | Optional
```

patient_logs

```
Field | Type | Notes
id | uuid | Primary key
phc_id | uuid | Foreign key
date | date | Log date
count | integer | Patient footfall
```

test_availability

```
Field | Type | Notes
```

```
id | uuid | Primary key
phc_id | uuid | Foreign key
blood_test | boolean | Available or not
xray | boolean | Available or not
ecg | boolean | Available or not
ultrasound | boolean | Available or not
updated_at | timestamp | Last update
```

alerts

```
Field | Type | Notes
id | uuid | Primary key
phc_id | uuid | Foreign key
alert_type | text | stock, bed, doctor, test, health_score
priority | text | low, medium, high, critical
title | text | Short title
message | text | Human-readable alert
recommendation | text | Suggested action
status | text | open, acknowledged, resolved
created_at | timestamp | Alert time
```

recommendations

```
Field | Type | Notes
id | uuid | Primary key
phc_id | uuid | Target PHC
type | text | medicine_transfer, patient_referral, doctor_assignment
priority | text | medium, high, critical
source_phc_id | uuid | Optional source PHC
payload | jsonb | Structured recommendation details
explanation | text | AI or system explanation
status | text | pending, accepted, rejected, completed
created_at | timestamp | Created time
```

reports

```
Field | Type | Notes
id | uuid | Primary key
report_type | text | daily, weekly, monthly
language | text | en, hi, gu
content | text | Generated report
file_url | text | Optional exported PDF
created_at | timestamp | Created time
```

9. Health Score Model

Use a transparent score from 0 to 100. Avoid hiding everything inside Gemini.

Recommended weights:

```
Factor | Weight
Medicine stock safety | 30
Doctor availability | 25
Bed availability | 20
Patient load pressure | 15
```

```
Test availability | 10
```

Example scoring logic:

```
health_score =
  medicine_score * 0.30 +
  doctor_score * 0.25 +
  bed_score * 0.20 +
  patient_score * 0.15 +
  test_score * 0.10
```

Risk bands:

```
Score | Status | Map Color
80-100 | Healthy | Green
60-79 | Watch | Yellow
40-59 | At Risk | Orange
0-39 | Critical | Red
```

10. Alert Engine

Run alert checks after every worker update and through a scheduled daily job.

Rules:

- Medicine stock below threshold: create high-priority stock alert.
- Medicine predicted to run out within 3 days: create critical alert.
- Bed occupancy above 90 percent: create high-priority bed alert.
- Bed occupancy above 98 percent: create critical alert.
- Doctor attendance below 70 percent for the week: create high-priority staffing alert.
- Health score below 60: create at-risk alert.
- Health score below 40: create critical intervention alert.
- Essential test unavailable for more than 2 days: create test availability alert.

Every alert should include:

- What happened.
- Which PHC/CHC is affected.
- Why it matters.
- Suggested action.
- Priority.

11. AI Features

11.1 Medicine Stock-Out Prediction

Best approach:

1. Use recent stock movements to calculate average daily consumption.
2. Estimate days until stock-out with a simple formula.
3. Ask Gemini to explain the risk and produce a clean recommendation.

Formula:

```
days_to_stockout = current_stock / average_daily_consumption
```

If stock history is missing, fall back to:

```
average_daily_consumption = max(1, patient_count_average * medicine_usage_factor)
```

Store output as structured JSON:

```
{
  "medicine": "Paracetamol",
  "phc": "PHC Naroda",
  "current_stock": 15,
  "days_to_stockout": 2,
  "priority": "critical",
  "recommendation": "Transfer 150 tablets from PHC Bapunagar"
}
```

11.2 Demand Forecast

For the MVP, use a simple forecast instead of a heavy ML model:

- 7-day moving average.
- Last 30-day trend.
- Day-of-week pattern.
- Seasonal multiplier.
- Optional Gemini explanation.

Inputs:

- Historical patient counts.
- Current season.
- Recent patient trend.
- Disease outbreak flag if available.
- Festival or local event note if manually entered.

Outputs:

- Tomorrow’s expected patients.
- Next 7-day expected patients.
- Surge risk: low, medium, high.
- Short reason.

11.3 Redistribution Optimizer

This is the highest-value feature.

Algorithm:

1. Identify target PHCs with low stock or high demand.
2. Find source PHCs with surplus stock.
3. Rank source PHCs by:

- Available surplus above threshold.
- Distance from target PHC.
- Same district or block.
- Source PHC’s own risk level.

4. Recommend transfer quantity without pushing source below safety threshold.

Transfer formula:

```
surplus = source_stock - source_threshold
target_gap = target_threshold - target_stock
transfer_quantity = min(surplus * 0.6, target_gap + 7_day_expected_usage)
```

Recommendation example:

```
Move 150 Paracetamol tablets from PHC Bapunagar to PHC Naroda.
Reason: PHC Naroda will run out in 2 days. PHC Bapunagar has 420 tablets above its safety
threshold.
```

11.4 Underperforming Center Detection

Use health score and repeated alerts to identify underperforming centers.

Flag a center if:

- Health score stays below 60 for 3 consecutive days.
- More than 5 critical alerts occur in 7 days.
- Doctor attendance stays below 60 percent in a week.
- Bed occupancy stays above 95 percent for 3 days.
- Two or more essential services are unavailable.

The admin dashboard should show reasons, not just a score.

11.5 AI Assistant

Use Gemini as a natural-language layer over structured district data.

Important rule:

Do not let the assistant guess. The backend should first query Supabase and pass only relevant structured context to Gemini.

Example prompt shape:

```
You are an assistant for a district health administrator.
Use only the provided JSON data.
Identify the top PHCs needing attention today.
Return concise recommendations with reasons.
```

Useful questions:

- Which PHCs need urgent attention today?
- Which medicines are likely to run out this week?
- Where should we move Paracetamol from?

- Which centers have poor doctor attendance?
- Generate today’s district health summary.

11.6 Multilingual Reports

Generate reports in:

- English.
- Hindi.
- Gujarati.

Report sections:

- District summary.
- Critical PHCs.
- Medicine alerts.
- Bed pressure.
- Doctor attendance.
- Test unavailability.
- Recommended actions.

12. API Design

Authentication

- ‘POST /auth/login‘
- ‘POST /auth/logout‘
- ‘GET /auth/me‘

If Supabase Auth is used directly in the frontend, the backend should still validate JWTs for protected APIs.

PHCs

- ‘GET /phcs‘
- ‘GET /phcs/{id}‘
- ‘GET /phcs/{id}/summary‘

Worker Updates

- ‘POST /medicines‘
- ‘PATCH /medicines/{id}‘
- ‘POST /beds/update‘
- ‘POST /doctor-attendance‘
- ‘POST /patient-logs‘
- ‘POST /test-availability‘

Admin Dashboard

- ‘GET /dashboard/district-summary‘
- ‘GET /dashboard/health-scores‘
- ‘GET /dashboard/patient-trends‘
- ‘GET /dashboard/map‘

Alerts

- ‘GET /alerts‘
- ‘PATCH /alerts/{id}/acknowledge‘
- ‘PATCH /alerts/{id}/resolve‘

AI and Recommendations

- ‘POST /ai/chat‘
- ‘POST /ai/generate-report‘
- ‘GET /recommendations‘
- ‘POST /recommendations/run‘
- ‘POST /forecasts/run‘

13. Frontend Pages

Public

- Login

Worker Portal

- Worker dashboard
- Medicine update page
- Bed update page
- Doctor attendance page
- Patient count page
- Test availability page

Admin Portal

- District overview dashboard
- AI command center
- PHC/CHC details
- Alerts
- Recommendations
- Reports
- AI assistant
- Settings

14. Admin Dashboard Layout

Top summary cards:

- District health score.
- Critical PHCs.
- Medicine alerts.
- Doctors absent.
- Available beds.
- Today’s patients.

Main panels:

- AI command center feed.
- District map with color-coded centers.
- Patient footfall trend.
- Medicine risk chart.
- Bed occupancy chart.
- Doctor attendance chart.
- Underperforming centers table.

PHC detail drawer:

- Health score.
- Current medicines at risk.
- Bed status.
- Doctor status.
- Test availability.
- Latest alerts.
- Recommended actions.

15. AI Command Center Feed

This should be the hero feature in the demo.

Each item should look like an operational instruction:

```
Critical: PHC Naroda
Paracetamol will run out in 2 days.
Recommended action: Transfer 150 tablets from PHC Bapunagar.
Reason: Bapunagar has surplus stock and is within 8 km.
```

Other examples:

```
High Risk: CHC Kalol
Bed occupancy is 98 percent.
Recommended action: Refer new non-critical patients to CHC Gandhinagar.


Staffing Alert: PHC Sanand
Doctor attendance is 58 percent this week.
Recommended action: Assign one temporary physician for the next 2 days.
```

16. Demo Data Strategy

Use generated data instead of real hospital data.

Seed:

- 50 PHCs/CHCs.
- 200-300 doctors.
- 50-100 medicines.
- 365 days of patient footfall.
- 30 days of stock movements.
- Random test availability gaps.
- Realistic latitude and longitude values.

Include intentionally risky centers:

- Low-stock PHC.
- Overcrowded CHC.
- Doctor absenteeism center.
- Test-unavailable center.
- High patient surge center.

This makes the demo predictable and impressive.

17. Development Roadmap

Phase 1: Foundation

- Create Next.js frontend.
- Create FastAPI backend.
- Create Supabase project and schema.
- Implement Supabase Auth.
- Add role-based routing.

Phase 2: Worker Portal

- Build data-entry forms for medicines, beds, doctors, patients, and tests.
- Connect forms to FastAPI.
- Validate and store updates.
- Trigger alert checks after updates.

Phase 3: Admin Dashboard

- Build district overview cards.
- Add charts with Recharts.
- Add Leaflet district map.
- Add PHC detail view.
- Add alerts table and filters.

Phase 4: Intelligence Layer

- Implement health score engine.
- Implement threshold alert engine.
- Implement stock-out forecast.
- Implement redistribution optimizer.
- Store recommendations.

Phase 5: Gemini Features

- Add AI assistant.
- Generate recommendation explanations.
- Generate daily/weekly reports.
- Add multilingual report output.

Phase 6: Demo Polish

- Generate realistic seed data.
- Add AI command center feed.
- Add voice input for worker updates.
- Add PDF export.
- Deploy frontend and backend.
- Prepare live demo script.

18. Suggested Folder Structure

```
smart-health-ai/
  frontend/
    app/
    components/
    lib/
    hooks/
    services/
    types/
    utils/
  backend/
    routers/
    models/
    schemas/
    services/
      gemini.py
      forecast.py
      alerts.py
      health_score.py
      redistribution.py
      reports.py
    database/
    utils/
    main.py
  seed/
    generate_demo_data.py
  docs/
    implementation.md
```

```
    demo-script.md
  README.md
```

19. Demo Script

Use this live demo sequence:

1. Login as district administrator.
2. Show district overview and map.
3. Open AI command center and show current alerts.
4. Login as health worker for a PHC.
5. Reduce Paracetamol stock from safe level to critical level.
6. Return to admin dashboard.
7. Show that:
- Alert appears.
- Health score drops.
- Map marker changes color.
- Stock-out prediction appears.
- Redistribution recommendation is generated.
8. Ask AI assistant: "Which PHCs need urgent help today?"
9. Generate a daily report in English and one Indian language.

This demonstrates the full story: live data, prediction, recommendation, and action.

20. Success Criteria

The implementation is successful if it can show:

- Real-time operational visibility across PHCs/CHCs.
- Early warnings before stock-outs and overloads.
- Clear health score for every center.
- Practical redistribution recommendations.
- AI assistant that answers district-level questions.
- Reports that administrators can use directly.
- A polished, reliable demo with seeded data.

21. Final Recommendation

Build the project as an AI decision-support platform, not a generic dashboard. Keep the core logic deterministic and transparent, then use Gemini to explain, summarize, translate, and assist. The winning implementation should make judges feel that the district administrator can see risk early and take action immediately.

The most important features to prioritize are:

1. AI command center feed.
2. Medicine stock-out prediction.
3. Redistribution optimizer.
4. District health score.

5. Map-based risk visualization.
6. AI assistant and report generation.

If time is limited, build fewer modules but make these features feel complete, connected, and demo-ready.